Scope of Improvement

1. Structured Intent

Current Situation

Your Acceptance Contract is written in plain English. This is easy for humans to review, but difficult for code to evaluate automatically.

Recommendation

Add a structured **expected_intent** block for important test cases. This allows the runner to compare the expected intent with the actual intent extracted from the SQL or graph state.

Why is this useful?

Instead of checking only whether the system produced an answer, you can verify whether it understood the user's business intent correctly.

2. Automatic Semantic Scoring

What does it mean?

Automatic semantic scoring compares the expected meaning of a question with what the backend actually produced. It compares intent, not exact SQL.

How to implement

Current flow:
Run Test → Capture Final Graph State → Write cases.jsonl

Improved flow:
Run Test → Capture Final Graph State → **Evaluate Semantic Score** → Write cases.jsonl

Example scoring

• Topic correct = 30
• Timeframe correct = 30
• Metric correct = 20
• Entity correct = 10
• Safe behavior = 10

Total = 100

Why is this important?

Your report can explain why a test passed or failed instead of simply reporting PASS. This gives measurable NL2SQL quality.

Some More Small Scope of Improvement

1. Track Accuracy Trends Over Time

Store and monitor:
• Pass rate
• Needs Review rate

• Average semantic score
• Top failing categories

Why?
You can measure whether every release improves or degrades the system instead of relying on individual test runs.

2. Add Latency and Cost Tracking

Capture for every test case:
• Runtime
• LLM calls
• BigQuery calls
• Tokens used

Why?
Enterprise NL2SQL systems must be accurate, but they must also be fast and cost-efficient.

3. Add Entity-Not-Found Cases

Example test cases:
• Show revenue for FakeCampaign123
• Compare Tuesday Promo BAU vs UnknownCampaign999

Expected behavior:
Return 'not found' or ask for clarification. Do not invent campaigns or results.